DAA FINAL PRESENTATION

Smart Cairo: Unified Optimization Framework

Routing analysis and fleet resource management for Cairo using graph theory, shortest-path search, traffic-aware weighting, greedy signal control, and dynamic programming.

A* Search · Dijkstra · Dynamic Programming · Emergency Routing

15	DISTRICT NODES MODELED
24	TRAFFIC-FLOW ROAD LINKS
10	BUS ROUTES IN PLANNING DATA

AIRPORT
NASR CITY
RAMSES HUB
GIZA
DOWNTOWN
HELWAN

PROJECT FOCUS
Urban mobility optimization
Route planning, emergency response, traffic signaling, and fleet allocation share the same graph model.

ACADEMIC SCOPE
DAA level implementation
Algorithms are implemented in Python with measurable complexity, tests, and simulation outputs.

ALAMEIN INTERNATIONAL UNIVERSITY | AI TRACK

01. PROBLEM STATEMENT

What the city model must solve

CAIRO TRANSPORT NETWORK

Congestion
Road weights change by time period, capacity, and road condition. A static shortest path is not enough for realistic routing.

Emergency routing
Ambulance paths must favor faster travel and priority intersections while still finding a valid hospital target.

Fleet allocation
Routes and buses are limited resources, so passenger coverage has to be optimized under capacity constraints.

Why this matters
- **Travel time** must adapt to real traffic profiles rather than distance alone.
- **Emergency response** needs priority-aware routing and intersection overrides.
- **Resource planning** must maximize passengers served per available vehicle or trip window.

Project objective
Unify routing, traffic, and allocation logic in one modular system so each algorithm solves a specific operational decision on the same transport graph.

GRAPH THEORY · EMERGENCY SUPPORT · GLOBAL OPTIMIZATION

PROBLEM FRAMED FROM THE PROCESSED CAIRO TRANSPORT DATA

02. PROJECT VISION

From raw CSV files to decision support

UNIFIED OPTIMIZATION

Vision
The system transforms district, road, demand, and facility data into routing and allocation decisions that can be evaluated, compared, and extended.

Inputs	Districts, roads, traffic, demand, facilities
Core models	TransportGraph, Edge, Node, TimePeriod
Outputs	Shortest paths, hospital routes, schedules, signal plans

What the project actually delivers
- **Dijkstra** for baseline shortest-route decisions.
- **A*** for heuristic-guided routing and emergency paths.
- **Time-dependent Dijkstra** for traffic-sensitive travel cost.
- **Prim** for infrastructure planning on road connections.
- **Greedy signal control** and emergency overrides for intersections.
- **DP solvers** for allocation and scheduling decisions.

SUPPORTED BY TESTS, SERVICES, AND SIMULATION MODULES

03. SYSTEM ARCHITECTURE

Modular pipeline with shared graph and service layers

LAYERED DESIGN

DataLoader · GraphBuilder · Routing Engine · Traffic Service · Planning Service · Simulation Output

Core layers
- **Models:** Node, Edge, TrafficProfile, TimePeriod, LocationType.
- **Graph:** TransportGraph stores adjacency and node metadata.
- **Algorithms:** shortest path, MST, greedy, and DP modules.

Services
- **RouteService** selects shortest or time-dependent routing.
- **EmergencyService** chooses the nearest reachable hospital and runs A*.
- **TrafficService** builds signal plans and congestion indices.
- **PlanningService** wraps Prim for expansion planning.

SAME GRAPH INTERFACE SHARED ACROSS ALL ALGORITHMS

04. GRAPH REPRESENTATION

TransportGraph with directed weighted edges

G(V, E)

NODE MODEL
Each node stores an ID, name, location type, coordinates, population, and adjacency references. The graph supports both district and facility nodes.

NODE(ID, NAME, TYPE, X, Y) · DISTRICT + FACILITY · POPULATION METADATA

EDGE MODEL
Edges store distance, capacity, road condition, and a traffic profile for morning, afternoon, evening, and night flows.

DIRECTED EDGE · DISTANCE(KM) · CAPACITY VEHICLES/HOUR · CONDITION 1-10

DISTRICT NODES
15
Maadi, Nasr City, Downtown Cairo, Helwan, Giza, and other districts in the processed dataset.

MEDICAL AND TRANSIT FACILITIES
10
Airports, hospitals, university nodes, rail hubs, and business anchors are represented as facility nodes.

ROAD LINKS
24+
Road geometry and traffic data are merged into a weighted transport graph for simulation and routing.

GRAPH STRUCTURE IS READ FROM PROCESSED CSVS AND TURNED INTO OBJECTS

05. DATASET AND CAIRO MODELING

Concrete dataset footprint from the repository

PROCESSED DATA

Processed inputs

FILE	ROLE	SCALE
neighborhoods_districts.csv	District nodes and coordinates	15 rows
existing_roads.csv	Road distance and capacity	24+ links
traffic_flow.csv	Time-period traffic volumes	24 links
transport_demand.csv	OD passenger demand	17 pairs
bus_routes.csv	Fleet allocation input	10 routes
metro_lines.csv	Transit infrastructure	3 lines
facilities.csv	Hospitals, airport, hubs	10 facilities

Cairo map modeling
- **Coordinates:** nodes are positioned using real x-y geographic coordinates from the processed data.
- **Demand centers:** hospitals, airport, university, railway station, and commercial hubs act as target points.
- **Traffic periods:** Morning Peak, Afternoon, Evening Peak, and Night are encoded in TimePeriod.
- **Road realism:** travel cost depends on distance, condition, and flow-capacity balance.

Dataset sources are aligned with the repository's processed CSV files and the supplied project data PDF.

COUNTS ARE TAKEN FROM THE CHECKED-IN PROCESSED FILES

06. DIJKSTRA ALGORITHM

Baseline shortest-path engine for static routing

SHORTEST PATH

```
dist[v] = min(dist[v], dist[u] + w(u, v))
```

- **Definition:** finds the least-cost path from one source to one destination on a non-negative weighted graph.
- **Why used:** provides the baseline route in the service layer and a fair comparison point for A*.
- **Role in project:** RouteService uses Dijkstra for the standard shortest route mode.
- **Complexity:** `O(E log V)` time and `O(V)` space in the implementation metadata.

Implementation details
- Uses a priority queue from **heapq**.
- Tracks explored nodes with **nodes_explored**.
- Consumes the shared **CostEvaluator** for edge weights.
- Reconstructs the final path from the predecessor map.

In benchmark mode the shared cost evaluator can keep the comparison fair by using static road distance.

SOURCE: BACKEND/ALGORITHMS/SHORTEST_PATH/DIJKSTRA.PY

07. A* SEARCH ALGORITHM

Heuristic-guided routing for faster emergency decisions

HEURISTIC SEARCH

```
f(n) = g(n) + h(n)
```

- **Definition:** balances the cost so far with a heuristic estimate to the nearest goal.
- **Heuristic used:** Euclidean distance to the closest goal position, scaled to stay admissible when priority discounts are present.
- **Role in project:** used by EmergencyService to route toward the nearest reachable medical facility.
- **Complexity:** `O(E log V)` time and `O(V)` space in the implementation metadata.

What the code tracks
- **Goal positions:** the graph stores facility coordinates for multi-goal routing.
- **nodes_explored:** explicit counter used in testing and evaluation.
- **Time awareness:** optional initial time feeds the shared cost evaluator.
- **Emergency support:** base emergency multiplier plus intersection priority scaling.

The heuristic is not a generic guess: it is the minimum Euclidean distance to any valid goal in the graph.

SOURCE: BACKEND/ALGORITHMS/SHORTEST_PATH/ASTAR.PY

08. COMPLEXITY ANALYSIS

As implemented in the repository

ASYMPTOTIC VIEW

MODULE	TIME COMPLEXITY	SPACE COMPLEXITY	ACADEMIC ROLE
Dijkstra	O(E log V)	O(V)	Baseline shortest route
A*	O(E log V)	O(V)	Heuristic-guided route search
Time-dependent Dijkstra	O(E log V)	O(V)	Traffic-sensitive routing
Prim	O(E log V)	O(V + E)	Infrastructure expansion planning
ResourceAllocationDP	O(n * capacity)	O(n * capacity)	0/1 knapsack allocation
SchedulingDP	O(n log n)	O(n)	Weighted interval scheduling

KEY CORRECTION
SchedulingDP is not knapsack
The implemented scheduling solver sorts intervals, uses binary search for compatibility, and performs weighted interval DP.

KEY CORRECTION
Dijkstra and A* share the same graph interface
The difference is the search strategy and heuristic guidance, not the data source or node model.

COMPLEXITIES TAKEN FROM ALGORITHM METADATA AND IMPLEMENTATION STRUCTURE

09. DYNAMIC TRAFFIC WEIGHT LOGIC

How the system turns traffic data into route cost

DYNAMIC COST MODEL

```
w(u,v) = d(u,v) * (1 + (flow/capacity)^2) * (1/condition)
```

```
cost = base_weight * emergency_multiplier * congestion_index
```

- **Time periods:** morning peak, afternoon, evening peak, and night are mapped from clock time.
- **Condition effect:** a lower road-condition score increases travel cost.
- **Congestion effect:** flow is normalized by capacity and squared to amplify heavy congestion.
- **Emergency effect:** default global multiplier is 0.8, with stronger intersection discounts via IntersectionPriority.

Why this is academically important
- **Realism:** dynamic weights reflect road condition and traffic variation instead of fixed distances.
- **Fair comparisons:** the shared evaluator can switch between benchmark and realistic modes.
- **Emergency routing:** the same framework can prioritize ambulance travel without rewriting the graph.

This is the actual cost logic used by Edge.get_weight() and CostEvaluator.edge_weight().

TIME-AWARE COSTS ARE CENTRAL TO THE ROUTING MODEL

10. FLEET ALLOCATION PROBLEM

Passenger coverage under limited vehicle capacity

RESOURCE PLANNING

Problem formulation
The repository models route allocation as a capacity-constrained optimization problem. Each route or trip has a weight and a passenger value, and the goal is to maximize total passengers covered.

WEIGHT = BUSES · VALUE = PASSENGERS · CAPACITY = FLEET LIMIT

- **ResourceAllocationDP** uses a classic 0/1 knapsack structure.
- **SchedulingDP** handles time-compatible trip selection separately.
- **TransitService** combines allocation and scheduling outputs for transit planning.

Real input examples

ROUTE	BUSES	PASSENGERS
B1	25	35000
B2	30	42000
B9	28	39000
B10	20	28000

These are repository dataset values from bus_routes.csv, so no fabricated allocation numbers are needed.

FLEET PLANNING USES THE DEMAND AND BUS-ROUTE PROCESSED FILES

11. 0/1 KNAPSACK SOLUTION

Exact DP for route selection and fleet usage

EXACT OPTIMIZATION

```
DP[i,w] = max(DP[i-1,w], v_i + DP[i-1,w-w_i])
```

- **Definition:** choose each route at most once while respecting total capacity.
- **Why used:** greedy selection can miss the global optimum when passenger value and bus usage conflict.
- **Implementation role:** ResourceAllocationDP builds a 2D DP table and reconstructs the chosen routes.
- **Complexity:** `O(n * capacity)` time and space in the current implementation.

What the code actually stores
- **weights:** bus counts per route, clamped to at least 1.
- **values:** passenger demand per route.
- **dp table:** rows are items, columns are capacity states.
- **output:** selected routes, total passengers covered, and a schedule dictionary.

SOURCE: BACKEND/ALGORITHMS/DP/RESOURCE_ALLOCATION.PY

12. DYNAMIC PROGRAMMING TABLE LOGIC

SchedulingDP is weighted interval scheduling

COMPATIBILITY DP

```
dp[i] = max(value_i + dp[p(i)], dp[i-1])
```

- **Correction:** the current implementation is not knapsack; it schedules non-overlapping trips.
- **Workflow:** parse times, create intervals, sort by finish time, binary-search the last compatible trip.
- **Role in project:** selects the best passenger-heavy trip subset without overlap.
- **Complexity:** `O(n log n)` time and `O(n)` space.

Why this matters
- It respects time compatibility instead of treating trips as independent items.
- Binary search keeps compatibility lookup efficient.
- Backtracking reconstructs the final schedule in route-by-route form.

This correction is important for academic accuracy because the file name can mislead readers.

SOURCE: BACKEND/ALGORITHMS/DP/SCHEDULING.PY

13. GREEDY VS DP COMPARISON

Speed of local choice versus global optimality

DECISION QUALITY

GREEDY
Traffic signal optimizer
Chooses the busiest incoming road at each intersection and assigns it the longest green phase. It is fast and suitable for live control.
- Local maximum by incoming flow.
- Used for real-time signal plans.
- Can miss a globally optimal citywide schedule.

DYNAMIC PROGRAMMING
Resource allocation and scheduling
Evaluates combinations or compatible intervals to maximize passenger coverage. It is slower, but the result is provably optimal for the model.
- Global optimum under constraints.
- Best for planning and allocation.
- Matches the academic objective of exact optimization.

Best use case
Greedy is the right fit for immediate traffic-light response, while DP is the right fit for offline planning when a full search is acceptable.

Project takeaway
The repository intentionally uses both approaches because the city problem contains both fast operational decisions and exact planning decisions.

DIFFERENT SUBPROBLEMS NEED DIFFERENT OPTIMIZATION STYLES

14. SYSTEM IMPLEMENTATION

Actual workflow from files to outputs

EXECUTION FLOW

STEP 1
Load processed CSVs
DataLoader normalizes IDs, merges road and traffic data, and creates Node and Edge objects.

STEP 2
Build TransportGraph
GraphBuilder creates the shared adjacency structure used by routing and planning modules.

STEP 3
Run services
RouteService, TrafficService, PlanningService, and EmergencyService invoke the right algorithm for the task.

STEP 4
Simulate and report
Simulation scripts export markdown summaries, CSV results, and plots for comparison.

Core files in the workflow
Backend/utils/data_loader.py · Backend/graph/graph_builder.py · Backend/services/route_service.py · Backend/services/emergency_service.py

Supporting modules
TrafficService · PlanningService · Performance evaluation · Simulation outputs

IMPLEMENTATION IS SERVICE-ORIENTED AND TESTABLE

15. TECHNOLOGIES USED

Actual stack used in the repository

TOOLS AND LIBRARIES

Core language
- **Python** for algorithms, services, models, and simulation.
- **HTML/CSS/JS** for presentation and the frontend shell.

Data and ML
- **Pandas** for CSV loading and preprocessing.
- **NumPy**, **scikit-learn**, and **joblib** in the traffic forecasting service.

Analysis and testing
- **Pytest** for unit and integration tests.
- **Matplotlib** for analysis plots and report generation.

IMPORTANT CLARIFICATION
Traffic ML is forecasting, not a neural network claim
The service uses GradientBoostingRegressor with preprocessing, so the deck should not claim a deep-learning model.

FRONTEND STACK
Modern visual presentation
The presentation itself uses a dark futuristic theme, glassmorphism, gradient accents, and responsive layout rules.

TECHNOLOGIES ARE TAKEN FROM REQUIREMENTS.TXT AND SOURCE IMPORTS

16. BENCHMARK RESULTS AND VALIDATION

Use only validated artifacts from the repository

VERIFIED OUTPUTS

Routing benchmark
Add Real Benchmark Here
- Use the latest A* vs Dijkstra runtime chart from performance evaluation.
- Do not present the checked-in CSV snapshot without regeneration because it conflicts with the analysis markdown.

Validation from tests
- **A* and Dijkstra** are asserted to return the same optimal cost on the synthetic test graph.
- **A* nodes explored** are expected to be less than or equal to Dijkstra in the test fixture.
- **ResourceAllocationDP** test cases verify optimal passenger coverage on sample inputs.

Emergency routing analysis
Add Real Benchmark Here
- analysis_report.md currently exists, but its numbers do not match the current CSV snapshot.
- Regenerate the report from fresh simulation outputs before the final defense.

Repository note: the checked-in response_time_comparison.csv currently shows zero reduction across rows, while analysis_report.md reports average and median reductions. Treat the report as stale until rerun.

VALIDATION IS REAL; SOME BENCHMARK OUTPUTS NEED REGENERATION

17. SCALABILITY AND OPTIMIZATION

How the system can grow without changing the core model

FUTURE-READY ARCHITECTURE

More districts
GraphBuilder and TransportGraph can scale to larger node sets as long as the processed CSVs are expanded consistently.

Better traffic intelligence
TrafficMLService can be used to forecast next-flow values, then feed calibrated congestion inputs into the cost evaluator.

Modular extensions
Additional routing policies or transit modes can be added without rewriting the graph layer or model classes.

Optimization directions
- Cache repeated route queries for common origin-destination pairs.
- Precompute or index critical hospitals and hubs for faster emergency lookup.
- Calibrate dynamic weights against updated traffic observations.

Algorithmic extensions
- Add multi-objective routing with safety or emissions as extra cost terms.
- Expand planning to larger MST or network-design scenarios.
- Integrate the simulation outputs into a live dashboard.

MODULAR DESIGN KEEPS FUTURE WORK REALISTIC

18. TEAM AND CREDITS

Project contributors extracted from the repository

CREDITS

TL **Basmala Elkady**
TEAM LEADER

A **Menna Abdo**

S **Mariam Ali**

M **Hoda Mahmoud**

M **Mohamed Ayman**

Credits and sources

Data source	Processed CSV files and Project_Provided_Data.pdf
Implementation source	Backend algorithms, services, models, and simulation scripts
Presentation design	Rebuilt as a dark futuristic academic deck

ADDITIONAL PROJECT MEMBERS WERE FOUND IN FRONTEND/INDEX.HTML

19. CONCLUSION AND FUTURE WORK

Clean academic summary and closing

FINAL SLIDE

Conclusion
- The project builds a shared transport graph for routing, emergency response, traffic control, and fleet planning.
- It combines exact optimization and heuristic search to address both planning and real-time decisions.
- The implementation is modular, testable, and grounded in the repository's actual data and code.

Future work
- Regenerate and publish final benchmark charts from the latest simulation outputs.
- Connect traffic forecasting more directly to routing cost updates.
- Expand the graph to more districts, facilities, and transit modes.
- Package the system with a live dashboard for decision makers.

THANK YOU

Q&A

Smart Cairo: Unified Optimization Framework

END OF PRESENTATION